

Longest Common Subsequence

Debarka Sengupta

Longest Common Subsequence

A **subsequence** of a string S , is a set of characters that appear in left-to-right order, but not necessarily consecutively.

Example

ACTTGCG

- *ACT* , *ATTC* , *T* , *ACTTGC* are all subsequences.
- *TTA* is not a subsequence

A **common subsequence** of two strings is a subsequence that appears in both strings. A **longest common subsequence** is a common subsequence of maximal length.

Example

$S_1 = AAACCGTGAGTTATTGTTCTAGAA$

$S_2 = CACCCCTAAGGTACCTTTGGTTC$

LCS is

ACCTAGTACTTTG

Has applications in many areas including biology.

Application: DNA Sequence Alignment

- ◆ Finding the best alignment between two DNA strings involves minimizing the number of changes to convert one string to the other.

```
X: ACCGGTCGAGTGC GCGGAAGCCGGCCGAA
    |  |  |  |  |  |  |  |  |  |  |  |
    G TC  GT  CG  G  AAGCCGGCCGAA
    GTCGT CGGAA GCCG  GC  C  G  AA
    |  |  |  |  |  |  |  |  |  |  |
Y:  GTCGTTCGGAATGCCGTTGCTCTGTAA
```

Figure 12.1: Two DNA sequences, X and Y, and their alignment in terms of a longest subsequence, GTCGTCGGAAGCCGGCCGAA, that is common to these two strings.

- ◆ A brute-force search would take exponential time, but we can do much better using **dynamic programming**.

Brute force algorithm

- For every subsequence of $X = \langle x_1, \dots, x_m \rangle$, check whether it is a subsequence of $Y = \langle y_1, \dots, y_n \rangle$, and record it if it is longer than the longest previously found.
- There are 2^m subsequences of X to check.
- For each subsequence, scan Y for the first letter. From there scan for the second letter, etc., up to the n letters of Y .
- Therefore, $\Theta(n2^m)$.

Theorem: Let $Z = \langle z_1, \dots, z_k \rangle$ be any LCS of $X = \langle x_1, \dots, x_m \rangle$ and $Y = \langle y_1, \dots, y_n \rangle$. Then

1. If $x_m = y_n$, then $z_k = x_m = y_n$, and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .
2. If $x_m \neq y_n$, then $z_k \neq x_m \Rightarrow Z$ is an LCS of X_{m-1} and Y .
3. If $x_m \neq y_n$, then $z_k \neq y_n \Rightarrow Z$ is an LCS of X and Y_{n-1} .

(1) can be proven by contradiction: if the last characters of X and Y are not included in Z , then a longer LCS can be constructed by adding this character to Z , a contradiction.

(2) and (3) have symmetric proofs: Suppose there exists a subsequence W of X_{m-1} and Y (or of X and Y_{n-1}) with length $> k$. Then W is a common subsequence of X and Y , contradicting Z being an LCS.

Optimal-substructure property of LCS problem

Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be sequences, and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any LCS of X and Y .

Then:

1) If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is LCS of X_{m-1} and Y_{n-1} .

$$X = \langle A, B, C, B, D, A, A \rangle \quad Y = \langle B, D, C, A, B, A \rangle \quad Z = \langle B, C, B, A \rangle$$

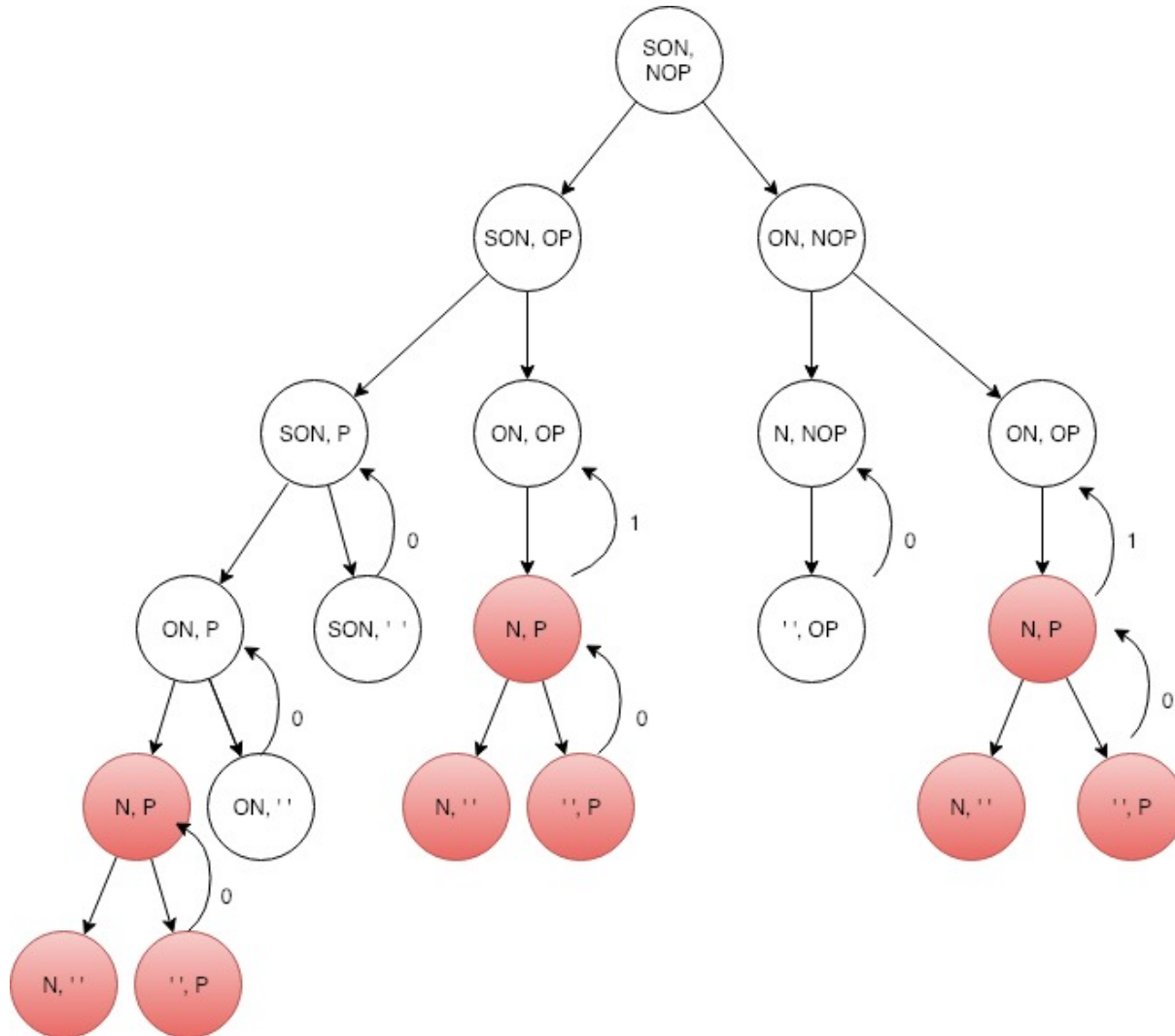
2) If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y .

$$X = \langle A, B, C, B, D, A \rangle \quad Y = \langle B, D, C, A, B \rangle \quad Z = \langle B, C, B \rangle$$

3) If $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1} .

$$X = \langle A, B, C, B, D, A, B \rangle \quad Y = \langle B, D, C, A, B, A \rangle \quad Z = \langle B, D, A, B \rangle$$

Overlapping sub-problem



Recursion

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

Using recursion

```
LCS(X,Y,m,n)
{
    if (m == 0 or n == 0)
        return 0
    else if (X[m-1] == Y[n-1])
        return (1 + LCS(X, Y, m-1, n-1))
    else
        return MAX(LCS(X, Y, m, n-1), LCS(X, Y, m-1, n))
}
```

Bottom up + back-tracking

LCS – Length(X, Y)

```
1   $m \leftarrow \text{length}[X]$ 
2   $n \leftarrow \text{length}[Y]$ 
3  for  $i \leftarrow 1$  to  $m$ 
4      do  $c[i, 0] \leftarrow 0$ 
5  for  $j \leftarrow 0$  to  $n$ 
6      do  $c[0, j] \leftarrow 0$ 
7  for  $i \leftarrow 1$  to  $m$ 
8      do for  $j \leftarrow 1$  to  $n$ 
9          do if  $x_i = y_j$ 
10             then  $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ 
11                  $b[i, j] \leftarrow \nwarrow$ 
12             else if  $c[i - 1, j] \geq c[i, j - 1]$ 
13                 then  $c[i, j] \leftarrow c[i - 1, j]$ 
14                      $b[i, j] \leftarrow \uparrow$ 
15                 else  $c[i, j] \leftarrow c[i, j - 1]$ 
16                      $b[i, j] \leftarrow \leftarrow$ 
17  return  $c$  and  $b$ 
```

Example

		0	1	2	3	4 = n
			B	D	C	B
0		0	0	0	0	0
1	B	0	1	1	1	1
2	A	0	1	1	1	1
3	C	0	1	1	2	2
4	D	0	1	2	2	2
$m=5$	B	0	1	2	2	3

$X = \text{BACDB}$
 $Y = \text{BDCB}$
 $\text{LCS} = \text{BCB}$

LCS Length Table

		0	1	2	3	4 = n
			B	D	C	B
0		0	0	0	0	0
1	B	0	1	1	1	1
2	A	0	1	1	1	1
3	C	0	1	1	2	2
4	D	0	1	2	2	2
$m=5$	B	0	1	2	2	3

with back pointers included

start here

Figure 32: Longest common subsequence example.